Développement Mobile avec Dart & Flutter

Chapitre 1 : Introduction au Développement Mobile (Partie 1/3)

Master Sciences et Techniques

Faculté des Sciences et Techniques

Année Universitaire 2025–2026

Objectifs du Chapitre

Grandes Orientations

- Comprendre les spécificités du paradigme mobile.
- Identifier les contraintes matérielles et logicielles des smartphones.
- Analyser l'écosystème Android et iOS.
- Comparer le développement natif, hybride et cross-platform.
- Découvrir le framework Flutter et le langage Dart.
- Comprendre l'architecture interne et la logique des Widgets.

1.1 Qu'est-ce qu'une application ?

Définition générale :

- Un programme informatique exécutant un ensemble d'instructions.
- Conçu pour aider un utilisateur à accomplir une tâche spécifique.
- S'exécute au-dessus d'un système d'exploitation (OS).

Les grandes familles :

1. Applications Desktop (Windows, macOS, Linux).
2. Applications Web (s'exécutant dans un navigateur).
3. Applications embarquées (IoT, microcontrôleurs).
4. **Applications mobiles** (smartphones, tablettes, wearables).

1.2 Qu'est-ce qu'une application mobile ?

Définition spécifique

Une application logicielle spécialement conçue pour s'exécuter sur un appareil terminal mobile : smartphone, tablette ou montre connectée.

Différences fondamentales avec le Web classique :

- Intégration profonde avec le matériel : capteurs, caméra, GPS.
- Exécution locale : fonctionnement hors ligne possible.
- Distribution via des magasins d'applications dédiés.
- Cycle de vie de l'application strictement géré par l'OS.

1.3 Caractéristiques d'une application mobile

Interactivité directe :

- Manipulation tactile.
- Gestes multi-touch.
- Réponses haptiques (vibrations).

Contexte d'utilisation :

- Utilisation nomade.
- Sessions courtes et fréquentes.
- Interruptions constantes : appels, notifications.

Ressources partagées :

- Gestion rigoureuse de la batterie.
- Gestion dynamique de la mémoire RAM.
- Connectivité fluctuante : 4G, 5G, Wi-Fi.

Sécurité :

- Modèle de sandbox (isolation).
- Gestion explicite des permissions.

1.4 Smartphone vs Tablette

Critère	Smartphone	Tablette
Taille écran	4.7″ à 6.8″	8.0″ à 13.0″
Orientation	Majoritairement portrait	Majoritairement paysage
Usage	Une seule main	Deux mains / Posé
Connectivité	Cellulaire permanent	Wi-Fi prédominant
Interface	Densité basse	Layouts complexes





Implication développement : conception d'interfaces réactives (**Responsive Design**) capables d'adapter la disposition des composants.

1.5 Contraintes d'un appareil mobile

Développer sur mobile nécessite un changement de paradigme par rapport au Desktop.

1. **Écran réduit :** priorisation stricte de l'information (UX).
2. **CPU/GPU limité :** nécessité d'optimiser le rendu à 60 FPS ou 120 FPS.
3. **Gestion énergétique :** mise en veille ou limitation des traitements de fond.
4. **Volatilité du réseau :** perte de paquets et passages en mode hors ligne.
5. **Diversité du parc matériel :** fragmentation des tailles d'écran et des versions d'OS.

1.6 L'écran tactile et les gestes

L'absence de pointeur physique (souris) impose une gestion événementielle adaptée.

- **Tap / Double Tap :** action basique / sélection.
- **Long Press :** affichage de menus contextuels.
- **Drag / Scroll :** déplacement continu du contenu.
- **Swipe :** transition rapide entre pages ou suppression.
- **Pinch / Spread :** zoom avant / arrière.

Cible tactile

Une taille d'environ **48** $\times$ **48 dp** est généralement recommandée pour une zone tactile confortable.

1.7 Contrainte : mémoire RAM limitée

Le problème :

- L'OS mobile peut arrêter des applications afin de libérer de la mémoire.
- Risque d'erreurs d'insuffisance mémoire (*Out Of Memory – OOM*).

Bonnes pratiques de développement :

- Libérer les ressources inutilisées.
- Fermer les flux de données et WebSockets inutiles.
- Optimiser la taille des images en mémoire.
- Utiliser le chargement différé (*Lazy Loading*).
- Éviter les fuites mémoire (*Memory Leaks*).

1.8 Contrainte : la batterie

La consommation énergétique dépend fortement de l'utilisation des différents composants matériels.

Directives d'optimisation

- Éviter les requêtes réseau répétitives.
- Utiliser un cache local.
- Regrouper les tâches d'arrière-plan (*Batch Processing*).
- Arrêter la géolocalisation haute précision lorsqu'elle n'est plus nécessaire.
- Réduire l'utilisation des timers et boucles inutiles.

1.9 Connectivité et réseau

Une application mobile doit être conçue en tenant compte des problèmes de connectivité.

Principe Offline-First

L'application doit rester utilisable, autant que possible, même lorsque la connexion Internet est absente.

- **État réseau instable :** Wi-Fi $\rightarrow$ 4G/5G $\rightarrow$ zone blanche.
- **Architectures adaptées :**
  1. Cache local pour les données fréquemment consultées.
  2. File d'attente pour les données à synchroniser.
  3. Gestion explicite des erreurs et des timeouts HTTP.

1.10 Les capteurs mobiles

Les téléphones modernes disposent de nombreux capteurs matériels.

- **Accéléromètre :** mesure l'accélération linéaire.
- **Gyroscope :** mesure la rotation spatiale sur trois axes.
- **Magnétomètre :** permet notamment de construire une boussole numérique.
- **Capteur de proximité :** désactive l'écran lors d'un appel.
- **Capteur de luminosité :** permet l'adaptation de la luminosité de l'écran.
- **Biométrie :** empreinte digitale ou reconnaissance faciale.

1.11 Géolocalisation : GPS

Moteurs de localisation combinés :

1. **GPS / GNSS :** localisation basée sur les satellites.
2. **Wi-Fi / réseau cellulaire :** localisation généralement plus rapide et moins précise.

Aspects techniques :

- Demande explicite des permissions à l'utilisateur.
- Différence entre localisation au premier plan (*Foreground*) et en arrière-plan (*Background*).
- Compromis entre précision et autonomie de la batterie.

1.12 Caméra et microphone

Utilisations :

- Capture multimédia.
- Réalité augmentée (AR).
- Scan de QR Codes.
- Enregistrement audio.
- Streaming.

Impacts sur le développement

- Déclaration explicite des permissions.
- Utilisation des APIs de la plateforme.
- Traitement des opérations lourdes hors du thread principal lorsque cela est nécessaire.
- Utilisation possible des *Isolates* avec Dart/Flutter.

1.13 Le système de notifications

Les notifications constituent un mécanisme important de communication avec l'utilisateur.

Notifications locales :

- Déclenchées localement.
- Basées sur un temps ou un événement.
- Exemple : rappel ou alarme.

Notifications push :

- Envoyées depuis un serveur distant.
- Firebase Cloud Messaging (FCM).
- APNs pour l'écosystème Apple.

1.14 Stockage local

Le stockage local permet de conserver des données sur l'appareil.

- **Key-Value Store :** préférences simples.
- **Base de données relationnelle :** SQLite, Room, sqflite.
- **Base NoSQL / Document :** Hive, Isar.
- **Fichiers :** images, PDF, documents et caches.

2.1 Panorama des systèmes mobiles

Le marché mobile est principalement dominé par deux grands écosystèmes : Android et iOS.

Android

- Projet Open Source Android (AOSP).
- Basé sur le noyau Linux.
- Grande diversité de constructeurs.
- Samsung, Xiaomi, Google, etc.

iOS

- Système propriétaire.
- Écosystème contrôlé par Apple.
- Disponible sur les appareils Apple compatibles.
- Forte intégration matériel / logiciel.

2.2 Android : historique et architecture

- **2003 :** création d'Android Inc.
- **2005 :** acquisition par Google.
- **2008 :** sortie de la première version commerciale d'Android.

Composants clés de l'architecture :

1. Noyau Linux.
2. Couche d'abstraction matérielle (HAL).
3. Android Runtime (ART).
4. Framework d'application.
5. Applications.

2.3 iOS : historique et écosystème

- **2007 :** lancement du premier iPhone.
- **2008 :** ouverture de l'App Store.
- **2010 :** adoption officielle du nom iOS.

Caractéristiques de l'écosystème :

- Intégration verticale matériel / logiciel.
- Contrôle important du système.
- Standards d'interface stricts.
- Human Interface Guidelines (HIG).

2.4 Comparatif Android vs iOS

Dimension	**Android**	**iOS**
Philosophie	Ouvert et personnalisable	Fermé et contrôlé
Langages natifs	Java / Kotlin	Swift / Objective-C
IDE officiel	Android Studio	Xcode
Fragmentation	Élevée	Plus faible
Publication	Flexible	Processus de validation strict

2.5 Distribution des applications : Google Play

Caractéristiques principales :

- Compte développeur Google Play.
- Analyse et vérifications avant publication.
- Publication sous forme d'Android App Bundle (**.aab**).
- Possibilité de distribuer directement des APK dans certains scénarios.

2.6 Distribution des applications : Apple App Store

Caractéristiques principales :

- Programme Apple Developer.
- Processus de validation des applications.
- Respect des règles de l'App Store.
- Distribution sous forme d'application iOS.
- Publication depuis l'écosystème Apple.

3.1 Les grandes familles d'architectures

Face aux différents systèmes mobiles, trois grandes approches de développement sont couramment étudiées :

1. **Développement natif :** une application spécifique par plateforme.
2. **Développement hybride :** technologies Web exécutées dans une WebView.
3. **Développement cross-platform :** une base de code permettant de cibler plusieurs plateformes.

3.2 Vue d'ensemble des approches

Natif
Code spécifique
Performances maximales

Hybride
Web + WebView
Développement rapide

Cross-Platform
Code partagé
Plusieurs plateformes

4.1 Le développement natif Android

Écosystème Android natif :

- **Langage historique :** Java.
- **Langage moderne :** Kotlin.
- **IDE :** Android Studio.
- **Interface utilisateur :**
  - Approche traditionnelle : Views et XML.
  - Approche moderne : **Jetpack Compose**.

4.2 Le développement natif iOS

Écosystème iOS natif :

- **Langage historique :** Objective-C.
- **Langage moderne :** Swift.
- **IDE :** Xcode.
- **Interface utilisateur :**
  - Approche traditionnelle : UIKit.
  - Approche moderne : **SwiftUI**.

4.3 Avantages et limites du natif

Avantages

- **Performances élevées**.
- Accès direct aux APIs de la plateforme.
- Accès rapide aux nouvelles fonctionnalités.
- UI conforme aux recommandations de la plateforme.

Limites

- Coût de développement élevé.
- Technologies différentes pour Android et iOS.
- Maintenance de plusieurs bases de code.
- Risque de duplication de la logique métier.

5.1 Le principe du Cross-Platform

Objectif :
Écrire une base de code commune et la déployer sur plusieurs plateformes :

- Android
- iOS
- Web
- Desktop

Bénéfices clés

- Réduction du temps de développement.
- Maintenance simplifiée.
- Réutilisation de la logique métier.
- Cohérence de l'application entre les plateformes.

5.2 React Native

Principe :

- Développement en JavaScript / TypeScript avec React.
- Utilisation de composants natifs.
- Communication entre le code JavaScript et les fonctionnalités natives de la plateforme.
- Architecture moderne basée notamment sur JSI et les mécanismes de communication natifs.

5.3 Flutter

Principe :

- Utilise le langage **Dart**.
- Fournit son propre système de widgets.
- Utilise un moteur de rendu graphique.
- Permet de construire des interfaces cohérentes sur plusieurs plateformes.
- Fournit notamment les bibliothèques **Material** et **Cupertino**.

Idée fondamentale

Avec Flutter, l'interface est construite à partir d'une hiérarchie de **Widgets**.

5.4 Grand tableau comparatif

Critère	Android natif	iOS natif	React Native	Flutter
Langage	Kotlin / Java	Swift	JS / TypeScript	Dart
Rendu UI	Composants natifs	Composants natifs	Composants natifs	Moteur Flutter
Code partagé	Faible	Faible	Élevé	Élevé
Hot Reload	Partiel	Partiel	Oui	Oui
Plateformes	Android	iOS	Android / iOS / Web	Android / iOS / Web / Desktop

Développement Mobile avec Dart & Flutter

Chapitre 1 : Introduction au Développement Mobile (Partie 1/3)

Master Sciences et Techniques

Faculté des Sciences et Techniques

Année Universitaire 2025–2026

1.1 Rappel : L'Application Mobile

Définition Synthétique

Logiciel exécuté sur un système d'exploitation mobile (Android/iOS) interagissant directement avec du matériel contraint (écran tactile, batterie, capteurs, réseau instable).

- **Objectif :** Offrir une expérience fluide (60 à 120 FPS).
- **Contexte :** Sessions courtes, sollicitations fréquentes, interruptions constantes.
- **Impératif :** Gestion rigoureuse des ressources matérielles.

1.2 Rappel : Duopole Android et iOS

Android (Google)

- Système ouvert basé sur Linux.
- Langages : Java, Kotlin.
- Rendu via les vues natives Android.
- Grande diversité de périphériques.

iOS (Apple)

- Écosystème fermé propriétaire.
- Langages : Objective-C, Swift.
- Rendu via UIKit / SwiftUI.
- Matériel strictement contrôlé.

1.3 Rappel : Natif vs Cross-Platform

Dimension	Développement Natif	Développement Cross-Platform
Code source	Deux dépôts distincts	Un seul dépôt partagé
Équipes	Spécialistes Android + iOS	Équipe unifiée
Coût / Temps	Doublé	Fortement optimisé
Maintenance	Risque de divergence	Logique centralisée

Le défi historique du Cross-Platform

Conserver des performances proches du natif tout en partageant le maximum de code d'interface et de logique.

1.4 Rappel : Positionnement de Flutter

- Flutter est le toolkit UI multiplateforme de Google.
- Contrairement aux approches basées sur une *WebView* ou un pont vers les composants natifs, Flutter **dessine lui-même ses pixels**.
- Il garantit une UI uniforme et des performances natives sur Android, iOS, Web et Desktop.

2.1 Définition et Origine de Dart

Qu'est-ce que Dart ?

Dart est un langage de programmation orienté objet, structuré par classes, développé par Google et dévoilé en 2011.

- **Créateurs :** Lars Bak et Kasper Lund (pionniers des moteurs V8 JavaScript et Smalltalk).
- **But initial :** Remplacer ou améliorer JavaScript pour les applications Web complexes.
- **Évolution :** Redestiné avec succès comme langage pivot du framework Flutter.

2.2 Objectifs du Langage Dart

1. **Productivité du développeur :** Syntaxe claire, familiarité immédiate pour les pratiquants de Java, C# ou C++.
2. **Performances d'exécution :** Capacité de compilation AOT (Ahead-Of-Time) pour une exécution ultra-rapide sur mobile.
3. **Souplesse de développement :** Compilation JIT (Just-In-Time) permettant le rechargement à chaud (*Hot Reload*).
4. **Rendu fluide :** Garanti par un gestionnaire de mémoire (Garbage Collector) optimisé pour la création/destruction massive d'objets éphémères.

2.3 Caractéristiques Clés de Dart

- **Orienté Objet Pur :** Tout concept manipulé est un objet (même les nombres et fonctions).
- **Typage Fort et Sûr :** Validation au moment de la compilation.
- **Null Safety Strict :** Prévention native des erreurs de type *Null Pointer Exception* à la compilation.
- **Mono-threadé par défaut :** Gestion de l'asynchronisme via une boucle d'événements (*Event Loop*) et des *Isolates*.

2.4 Syntaxe Générale de Dart

Exemple synthétique :

Structure d'un programme Dart

```
void main() {
  String nom = 'Master ST';
  int annee = 2026;
  print('Bienvenue en $nom ($annee)');
}
```

- Point d'entrée obligatoire : fonction `main()`.
- Manipulation de chaînes avec interpolation (`$variable`).
- Syntaxe familière terminée par des points-virgules.

2.5 Système de Typage dans Dart

Typage Statique :

- Les types sont vérifiés à la compilation pour éviter les bugs à l'exécution.

Inférence de Type :

- Le mot-clé `var` laisse Dart déduire automatiquement le type selon la valeur d'initialisation.

Immuabilité :

- `final` : Valeur assignée une seule fois à l'exécution.
- `const` : Constante absolue calculée dès la compilation.

2.6 Compilations JIT et AOT dans Dart

La puissance de Dart réside dans son architecture de compilation double :

Compilation JIT (Just-In-Time)

- Le code est compilé pendant l'exécution.
- Utilisé en **Phase de Développement**.
- Permet le **Hot Reload** (injection de code en < 1 seconde).

Compilation AOT (Ahead-Of-Time)

- Le code est compilé en **code machine natif** (ARM/x86) avant l'exécution.
- Utilisé en **Production**.
- Démarrage instantané et performances maximales.

2.7 Pourquoi Flutter utilise-t-il Dart ?

Google a choisi Dart pour Flutter pour 4 raisons stratégiques :

1. **Support JIT + AOT :** Seul langage offrant simultanément le développement ultra-rapide (JIT) et des binaire natifs performants (AOT).
2. **Garbage Collection pour interfaces réactives :** Gestion optimale des milliers de widgets détruits et recréés chaque seconde sans saccade (*jank*).
3. **Langage déclaratif lisible :** Permet la composition d'interfaces en arbre sans syntaxe de balisage séparée (pas besoin de XML ou HTML).
4. **Contrôle total de l'équipe :** Dart et Flutter sont développés en synergie chez Google.

3.1 Définition de Flutter

Définition

Flutter est un UI Toolkit (boîte d'outils d'interface utilisateur) open-source créé par Google pour concevoir des applications multiplateformes compilées nativement à partir d'un seul code source.

- Ce n'est pas un simple "wrapper" HTML/JS.
- Ce n'est pas un générateur de vues natives OS.
- C'est un moteur complet incluant son propre système de rendu, de composants et d'outils.

3.2 Flutter : Toolkit vs Framework

En tant que Framework :

- Fournit une architecture logicielle structurée.
- Impose un modèle orienté objets et déclaratif.
- Gère l'arbre de composants et la logique de rendu.

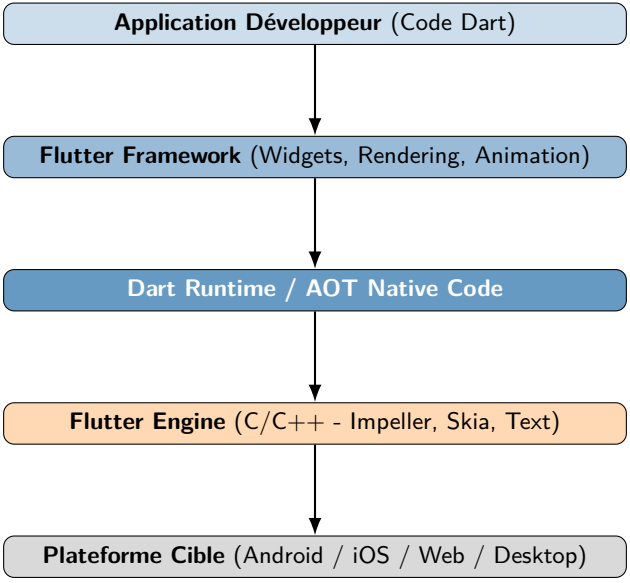
En tant que Toolkit :

- Inclut une vaste bibliothèque de Widgets (Material / Cupertino).
- Propose des outils de débogage avancés (*DevTools*).
- Intègre un moteur graphique natif optimisé.

3.3 Le Moteur de Rendu Graphique

- **Skia / Impeller :** Flutter utilise son propre moteur graphique 2D haute performance (Impeller sur les versions récentes d'iOS/Android).
- **Contrôle Pixel-Par-Pixel :** Flutter ne demande pas au système d'exploitation de dessiner un bouton : Flutter dessine le bouton directement dans une canvas native.
- **Résultat :** Une fidélité visuelle absolue et un comportement rigoureusement identique quelle que soit la version de l'OS.

4.1 Relation Conceptuelle Dart / Flutter



4.2 Rôles Respectifs

Séparation des Responsabilités

- **Dart** fournit le langage, la logique objet, la gestion de la mémoire, les structures de données et les fonctionnalités asynchrones.
- **Flutter** fournit la couche visuelle : la hiérarchie des Widgets, le calcul des dispositions (*Layout*), la gestion des thèmes et les animations.

5.1 La philosophie centrale de Flutter : "Everything is a Widget"

En Flutter, l'interface utilisateur globale est construite exclusivement par assemblage de **Widgets**. Un Widget en Flutter définit :

- Un élément visuel (Ex : `Text`, `Image`, `Icon`).
- Un élément de disposition / structure (Ex : `Row`, `Column`, `Center`, `Padding`).
- Un élément d'interaction (Ex : `GestureDetector`, `ElevatedButton`).
- Un élément de configuration ou de thème (Ex : `Theme`, `MediaQuery`).

5.2 Composition vs Héritage

Flutter privilégie la **Composition** plutôt que l'Héritage :

- Au lieu de créer un bouton complexe en héritant d'une classe de base et en surchargeant 50 méthodes, on **compose** plusieurs petits widgets simples.
- Ex : Un bouton personnalisé = Un widget `GestureDetector` contenant un `Container` (style), contenant un `Row` (alignement), contenant une `Icon` et un `Text`.

5.3 Relations Parent-Enfant

Les widgets s'imbriquent les uns dans les autres grâce à des propriétés dédiées :

Propriété `child`

Utilisée lorsqu'un widget accepte **un seul enfant**.

- Ex : `Center`, `Padding`, `SizedBox`, `Container`.

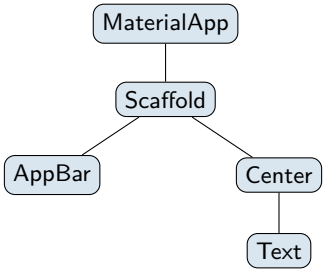
Propriété `children`

Utilisée lorsqu'un widget accepte **plusieurs enfants** (`List<Widget>`).

- Ex : `Row`, `Column`, `Stack`, `ListView`.

6.1 L'Arbre de Widgets (Widget Tree)

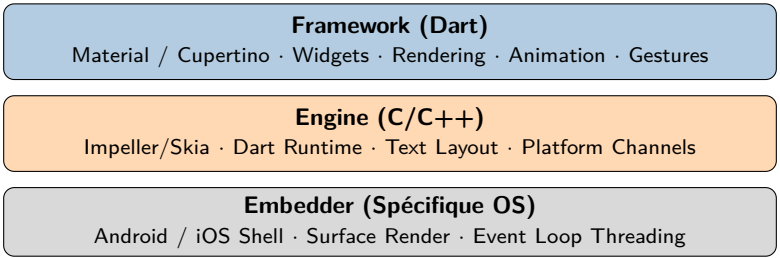
L'imbrication des widgets forme une structure arborescente appelée **Widget Tree**.



6.2 Explication Niveaux par Niveau

- **MaterialApp :** Racines de l'application définissant le thème global, la navigation et la localisation.
- **Scaffold :** Structure de page standard suivant le Material Design (fournit des emplacements pour AppBar, Body, Drawer, etc.).
- **AppBar :** Barre supérieure d'en-tête de la page.
- **Center :** Widget de disposition centrant son contenu sur l'espace disponible.
- **Text :** Widget terminal affichant une chaîne de caractères stylisée.

7.1 Architecture en Couches de Flutter



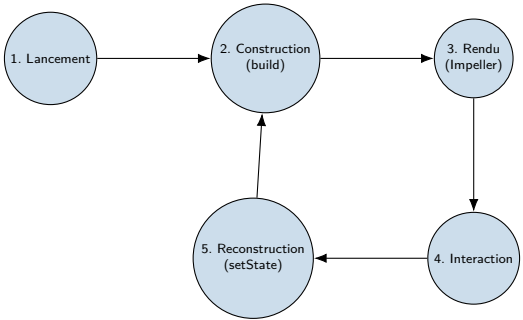
7.2 Le Rôle de l'Embedder

Rôle stratégique de l'Embedder

L'Embedder est la couche d'adaptation pointant vers la plateforme hôte (Android, iOS, Windows, macOS, Linux, Web).

- Il initialise le fil d'exécution principal (*Main Thread*).
- Il crée la surface d'affichage native (ex : `SurfaceView` sous Android, `CAMetalLayer` sous iOS).
- Il traduit les événements système (touches tactiles, clavier) en événements Flutter.

8.1 Le Cycle de Vie d'une Interface Flutter



8.2 Introduction aux Types de Widgets

StatelessWidget

- **Immuable** (Ses propriétés ne changent pas).
- Dépend uniquement des informations reçues lors de sa construction.
- Reconstruit uniquement si son parent change.

StatefulWidget

- Possède un état interne **mutable** associé (`State`).
- Peut changer d'apparence en cours d'exécution suite à une action utilisateur.
- Se re-dessine en appelant la méthode `setState()`.

9.1 Le Hot Reload (Rechargement à Chaud)

Principe

Grâce à la VM Dart en mode JIT, le code modifié par le développeur est injecté dans la machine virtuelle en cours d'exécution **sans redémarrer l'application**.

- **Conservation de l'état (Stateful) :** Si vous êtes sur un écran précis au 5ème niveau de navigation avec des données saisies, le Hot Reload met à jour le visuel sans perdre vos données en cours !
- **Gain de temps :** Passage d'un cycle de recompilation de 1 à 2 minutes en natif à **moins d'une seconde** en Flutter.

9.2 Hot Reload vs Hot Restart

Option	Hot Reload	Hot Restart
Temps d'exécution	$<$ 1 seconde	2 à 5 secondes
Conservation de l'État	**Oui**	Non (Réinitialisation)
Appel de `main()`	Non	Oui
Usage principal	Ajustements UI / CSS-like	Changement de logique `main` / State global

10.1 Synthèse Critique : Avantages de Flutter

- **Productivité Exceptionnelle :** Hot Reload + Code unique pour 6 plateformes.
- **UI Expressive et Flexible :** Pas de limitation imposée par les vues natives de l'OS.
- **Performances Proches du Natif :** Compilé AOT, rendu 60/120 FPS constant via Impeller.
- **Écosystème Riche :** Plus de 40 000 packages disponibles sur `pub.dev`.

10.2 Synthèse Critique : Limites de Flutter

Points d'attention et contraintes

- **Taille des Binaires :** L'inclusion du moteur Flutter (Engine) ajoute quelques mégaoctets au binaire final ($\sim 4 - 5$ Mo minimum).
- **Délai d'accès aux nouvelles API OS :** Une nouveauté système nécessite parfois un temps d'attente ou la création d'un plugin natif personnalisé via *Platform Channels*.
- **Dépendance envers Google :** Pérennité liée à la feuille de route stratégique de l'éditeur.

11.1 Exemple Conceptuel : Hello World Widget

Code minimal Flutter

```
import 'package:flutter/material.dart';

void main() {
  runApp(
    const MaterialApp(
      home: Scaffold(
        body: Center(
          child: Text('Bonjour Master ST !'),
        ),
      ),
    ),
  );
}
```

12.1 Questions de Compréhension (1/2)

1. Qui est le créateur du langage Dart et en quelle année a-t-il été dévoilé ?
2. Quelle est la différence entre compilation JIT et compilation AOT ?
3. Pourquoi le mode JIT est-il crucial pour le développement Flutter ?
4. Que signifie le slogan "Everything is a widget" dans Flutter ?
5. Pourquoi Flutter n'utilise-t-il pas les vues natives iOS et Android ?
6. Quel est le rôle du moteur graphique Impeller/Skia ?
7. Quelle est la différence entre la propriété `child` et `children` ?
8. Expliquer le rôle du widget `MaterialApp`.

12.2 Questions de Compréhension (2/2)

9. Qu'est-ce que le *Widget Tree* ?
10. Quel est le rôle de la couche *Embedder* dans l'architecture Flutter ?
11. Quelle est la différence fondamentale entre `Hot Reload` et `Hot Restart` ?
12. Que se passe-t-il au niveau de l'état de l'UI lors d'un Hot Reload ?
13. Citer deux avantages majeurs de la composition par rapport à l'héritage d'IHM.
14. Quelles sont les principales limites de Flutter ?
15. Qu'est-ce que le dépôt officiel de packages Dart/Flutter ?

13.1 Exercices Conceptuels (1/2)

Exercice 1 : Dessin de Widget Tree

- Représenter sous forme d'arbre l'IHM suivante : Une page contenant une barre d'en-tête, et au centre une colonne comprenant un texte et une icône.

Exercice 2 : Choix de Compilation

- Indiquer le type de compilation Dart utilisé lors de l'exécution de la commande `flutter run` en mode debug vs `flutter build apk -release`.

Exercice 3 : Identification de Propriétés

- Le widget `Column` accepte-t-il une propriété `child` ou `children` ? Justifier.

13.2 Exercices Conceptuels (2/2)

Exercice 4 : Analyse d'Architecture

- Placer les éléments suivants dans la bonne couche (Framework, Engine, Embedder) : *Impeller*, *Material Buttons*, *Shell Android*, *GestureRecognizer*, *Dart VM*.

Exercice 5 : Réparation Conceptuelle

- Un développeur modifie la méthode `main()` de son application. Le Hot Reload ne prend pas en compte la modification. Pourquoi ? Quelle action doit-il effectuer ?

14.1 Synthèse et Vocabulaire Essentiel

Termes Clés à Retenir

- **Dart :** Langage orienté objet compilable en JIT et AOT.
- **Widget :** Composant d'interface immuable de base.
- **Widget Tree :** Arbre hiérarchique décrivant la structure de l'IHM.
- **Hot Reload :** Injection rapide de modifications de code sans perte d'état.
- **Engine :** Moteur C++ dessinant les pixels directement sur la canvas native.

Prochaine Étape : Installation du SDK Flutter, configuration de l'IDE (Android Studio / VS Code) et écriture du premier projet mobile !